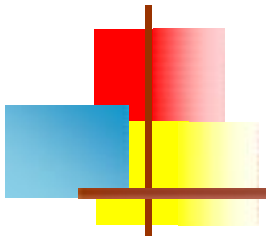


Classes and Objects in Java



Contents

- Introduce to classes and objects in Java.
- The three principles of OOP
- Understand how some of the OO concepts learnt so far are supported in Java.
- Understand important features in Java classes.

Classes and Objects

- A Java program consists of one or more classes
- A class is an abstract description of objects
- Here is an example class:
  - `class Dog { ...description of a dog goes here... }`
- Here are some objects of that class:



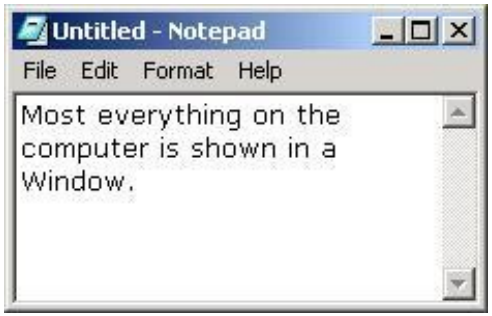


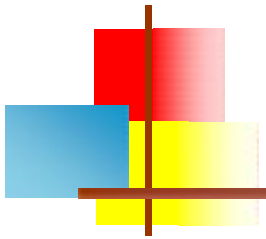

More Objects

- Here is another example of a class:
  - class Window { … }
- Here are some examples of Windows:









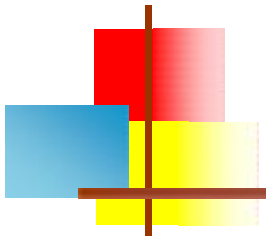
Classes contain data definitions

- Classes describe the data held by each of its objects
- Example:

Data usually goes first in a class

```
class Dog {
    String name;
    int age;
    ...rest of the class...
}
```

- A class may describe any number of objects
  - Examples: "Fido", 3; "Rover", 5; "Spot", 3;
- A class may describe a single object, or even no objects at all

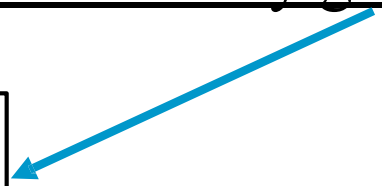


Classes contain methods

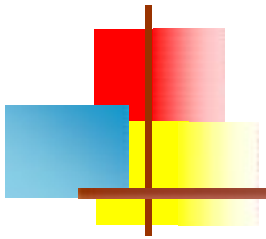
- A class may contain methods that describe the behavior of objects
- Example:

```
class Dog {
    ...
    void bark() {
        System.out.println("Woof!");
    }
}
```

Methods usually go after the data

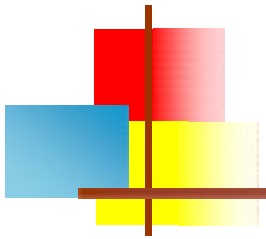


- When we ask a *particular* Dog to bark, it says “Woof!”
- Only Dog *objects* can bark; the *class* Dog cannot bark



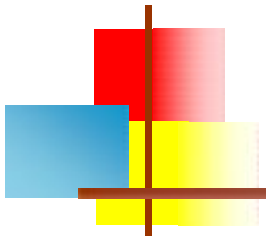
Methods contain statements

- A statement causes the object to do something
  - (A better word would be “command”—but it isn’t)
- Example:
  - `System.out.println("Woof!");`
  - This causes the particular `Dog` to “print” (actually, display on the screen) the characters `Woof!`



Methods may contain temporary data

- Data described in a class exists in all objects of that class
  - Example: Every Dog has its own name and age
- A method may contain local *temporary* data that exists only until the method finishes
- Example:
  -
```
void wakeTheNeighbors( ) {
    int i = 50;   // i is a temporary variable
    while (i > 0) {
        bark( );
        i = i - 1;
    }
}
```



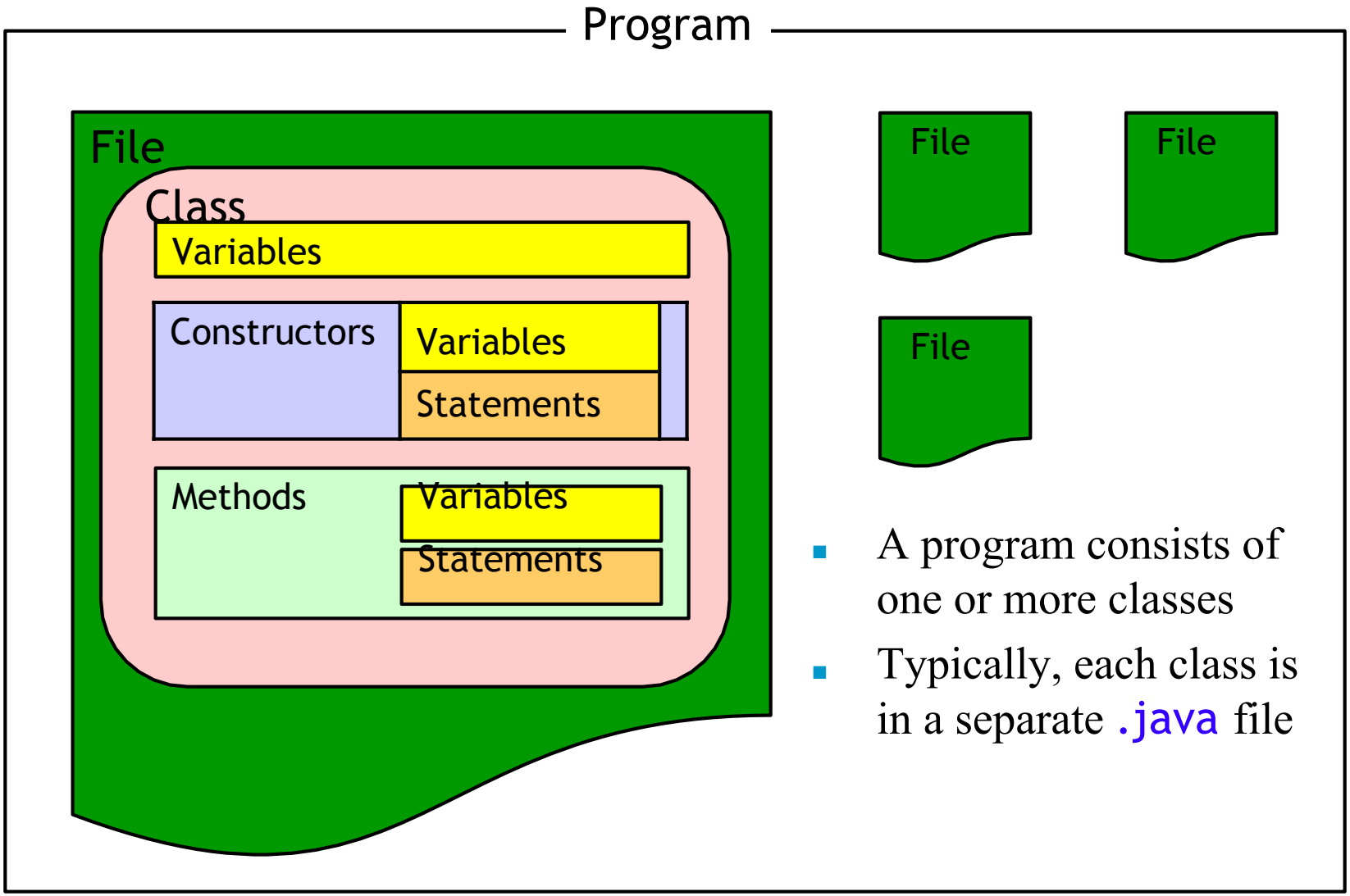
Classes always contain constructors

- A constructor is a piece of code that "constructs," or creates, a new object of that class
- If you don't write a constructor, Java defines one for you (behind the scenes)
- You can write your own constructors
- Example:

```
class Dog {
    String name;
    int age;
    Dog(String n, int age) {
        name = n;
        this.age = age;
    }
}
```

(This part is the constructor)

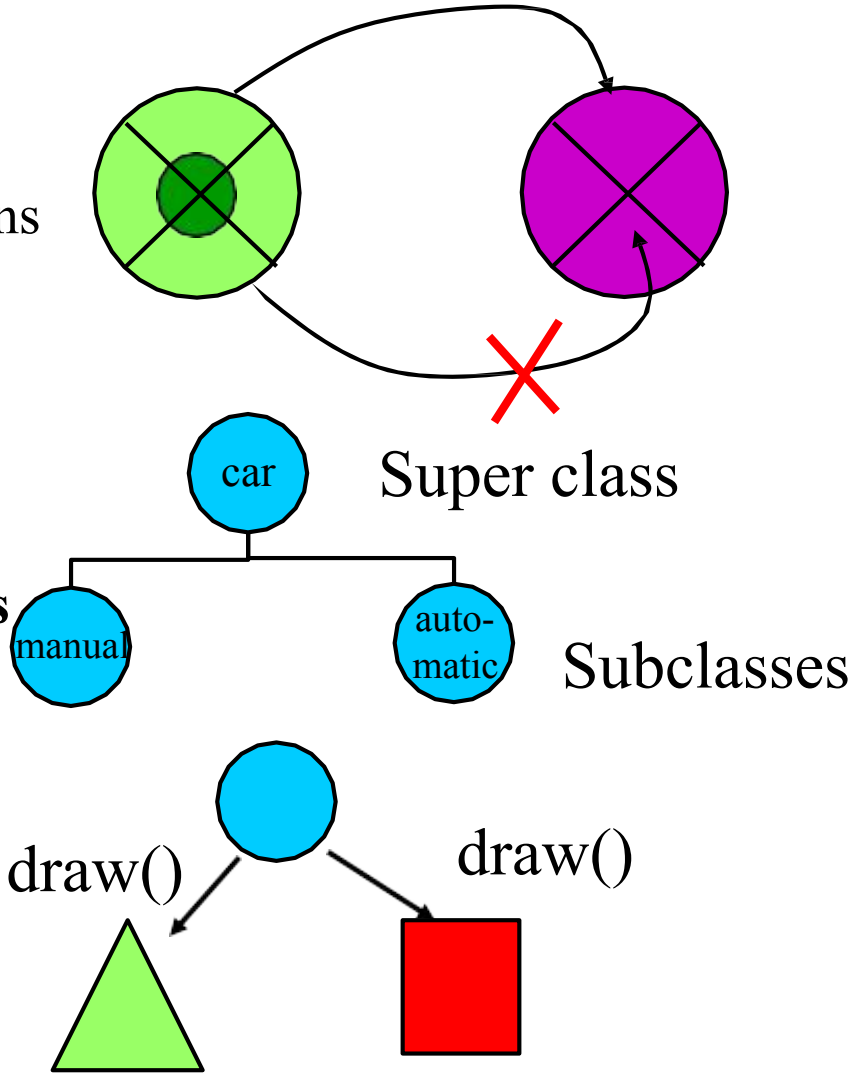
Diagram of program structure

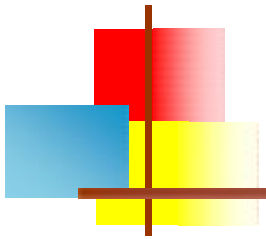


- A program consists of one or more classes
- Typically, each class is in a separate `.java` file

The three principles of OOP

- Encapsulation
  - Objects hide their functions (**methods**) and data (**instance variables**)
- Inheritance
  - Each **subclass** inherits all variables of its **superclass**
- Polymorphism
  - Interface same despite different data types



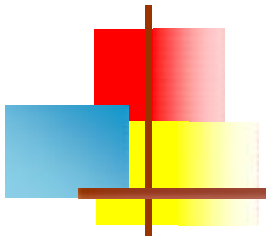


Java classes

- Java is a true OOP language, all the underlying structure of all programs is classes. Anything we wish to represent in a java program must be encapsulated in a class that defines the state and behavior of the basic programming components that is called object.
- Classes create objects and objects use methods to communicate between them.

- Defining a class:

```
class classname [extend superclassname]
{
   [variable
    declarations]
    [methods
    declarations]
}
```



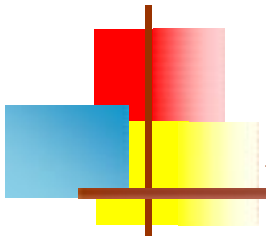
Java Classes

- A *class* is a collection of *fields* (data) and *methods* (procedure or function) that operate on that data.
- The basic syntax for a class definition:

```
class ClassName [extends SuperClassName]
{
        [fields declaration]
        [methods declaration]
}
```

- Bare bone class – no fields, no methods

```
public class Circle
{
    // my circle class
}
```

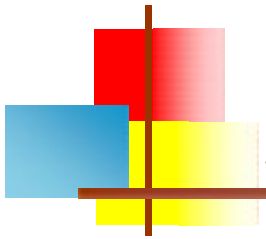


Adding Fields: Class Circle with fields

- Add *fields*

```
class Circle
{
        double radius;
}
```

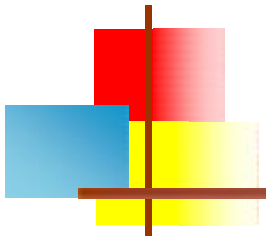
- The fields (data) are also called the *instance* variables.



Adding Methods

- A class with only data fields has no life. Objects created by such a class cannot respond to any messages.
- Methods are declared inside the body of the class but immediately after the declaration of data fields.
- The general form of a method declaration is:

```
class Circle
{
        double radius;
       double findArea
       ()
       {
               return radius * radius * 3.14159;
        }
}
```



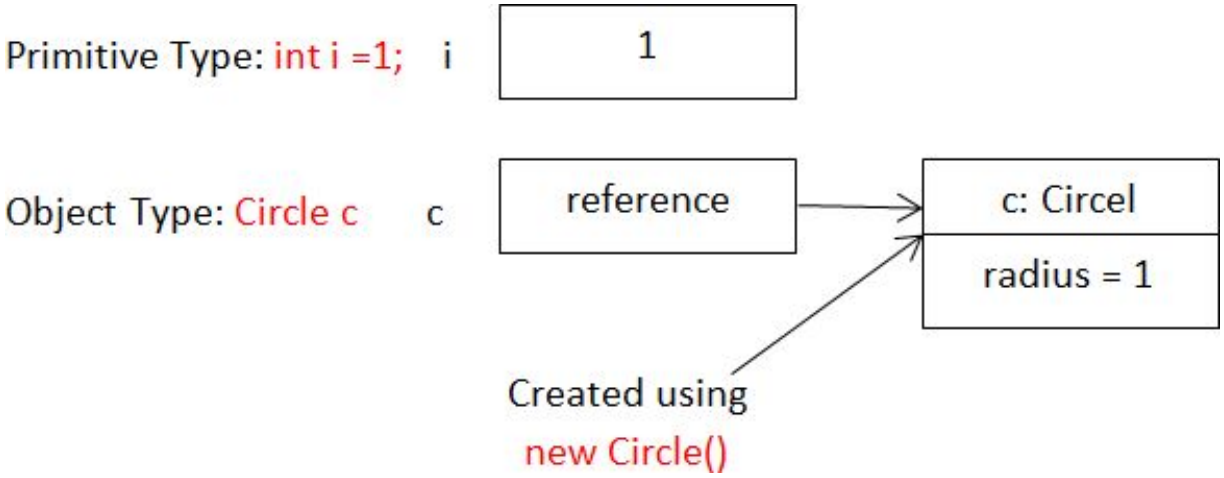
Creating java object

- An object in java is a block of memory that contains space to store all the instance variables. Creating an object is also referred to as instantiating an object.

- `Circle c;` // declare, no memory allocation at this point
- `c = new Circle();` // instantiate, allocate memory

- in a single step: `Circle c = new Circle();`

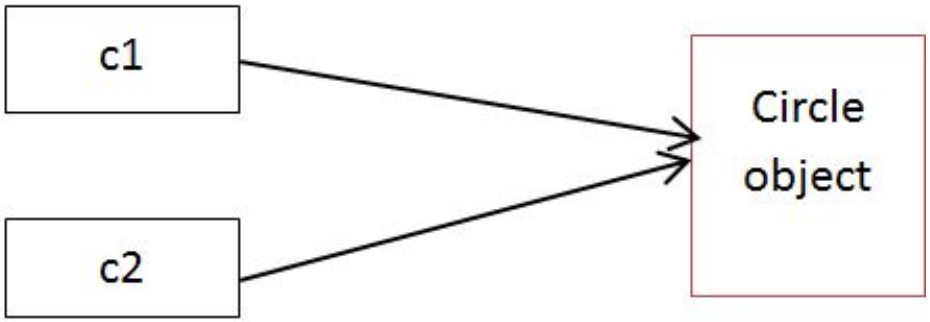
Differences between variables of primitive Data types and object types



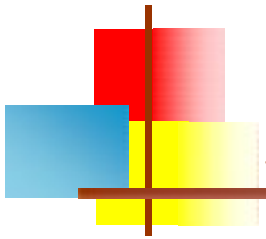
Assign object to another object

- `Circle c1 =new Circle();`
- `Circle c2 = c1`

Circle c1 =new Circle();
Circle c2 = c1



Both refer to the same memory location



Accessing Objects

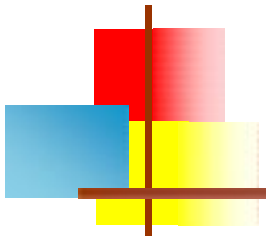
- `Circle myCircle =new Circle();`

- Referencing the object's data:

```
objectReference.data
myCircle.radius;
```

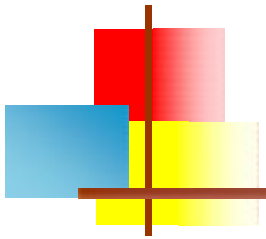
- Invoking the object's method:

```
objectReference.method
myCircle.findArea();
```



Constructors

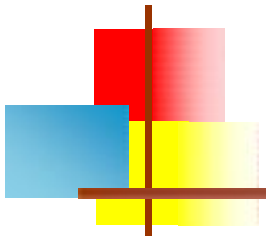
- Constructors are a special kind of methods that are invoked to construct objects.
- All object that are created must be an initial value. Question how we can fill-up the initial values to all the instance variables
  - Using dot operator: this is not a good approach
  - Better approach is provide all the initial values during the construction of an object.
- Java support a special type of method called a constructor, that initialize an object when it is first created.
- Rules:
  - Constructor have the same name as the class name
  - They do not specify any return type.



Constructors

```
class Circle
{
        double radius;
        Circle(double r)
        {
                radius = r;
        }
        Circle()
        {
                radius = 1.0;
        }
}

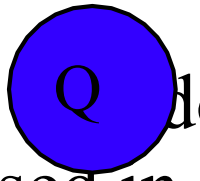
myCircle = new Circle(5.0);
```



Constructors (cont…)

- A constructor with no parameters is referred to as a *default constructor*.
- Constructors must have the same name as the class itself.
- Constructors do not have a return type—not even void.
- Constructors are invoked using the new operator when an object is created. Constructors play the role of initializing objects.

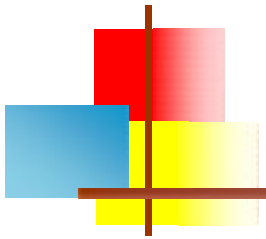
Automatic garbage collection



- The object Q does not have a reference and cannot be used in future.

- The object becomes a candidate for automatic garbage collection.

- Java automatically collects garbage periodically and releases the memory used to be used in the future.



Garbage Collection

- As shown in the previous figure, after the assignment statement c1 = c2, c1 points to the same object referenced by c2. The object previously referenced by c1 is no longer useful. This object is known as garbage. Garbage is automatically collected by JVM.
-
- TIP: If you know that an object is no longer needed, you can explicitly assign null to a reference variable for the object. The Java VM will automatically collect the space if the object is not referenced by any variable.



https://ecomputernotes.com/java/what-is-java-classes/calculate-area-of-rectangle-using-class

https://ecomputernotes.com/java/what-is-java-classes/area-of-rectangle-using-default-constructor



Thank you